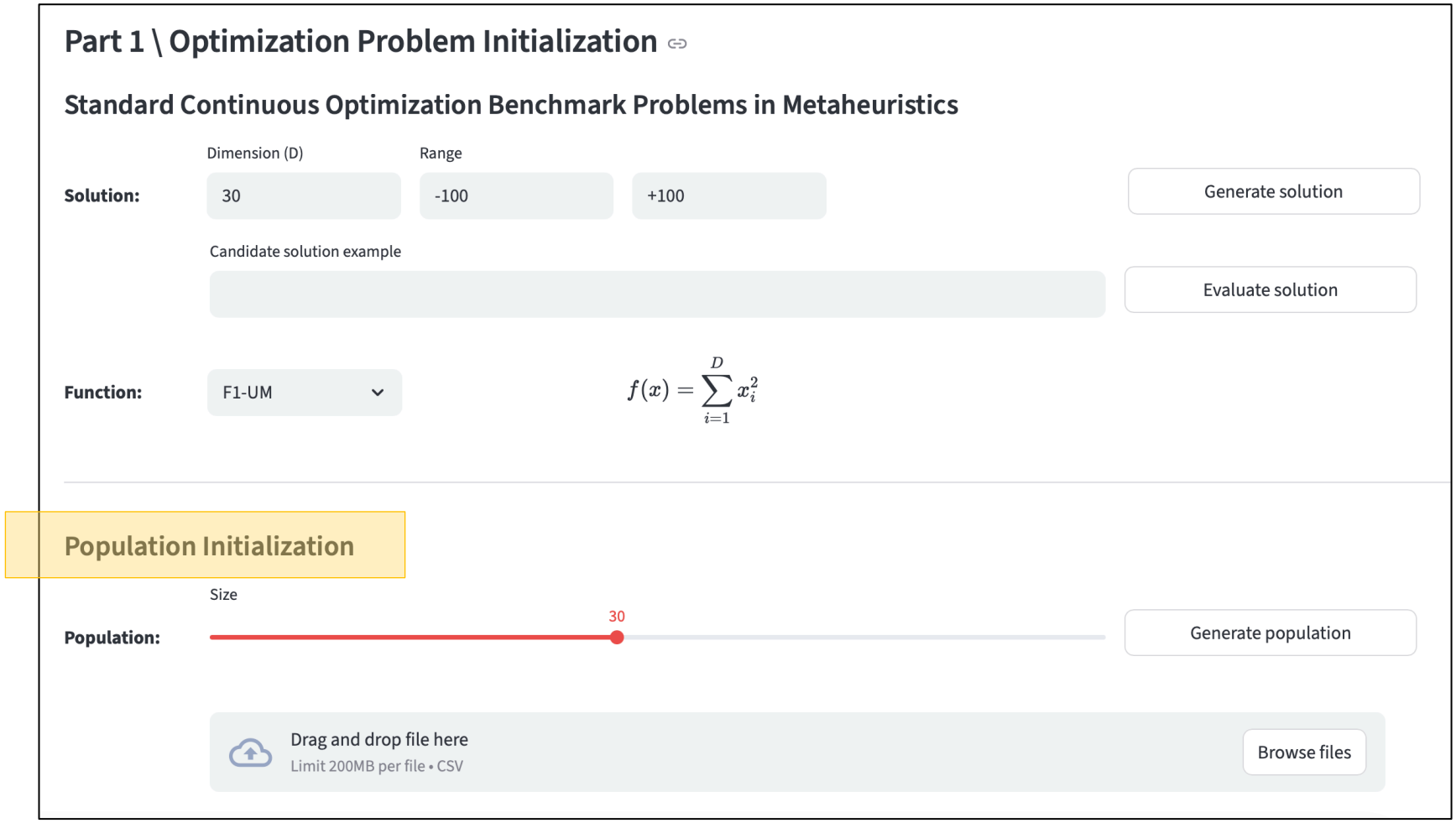
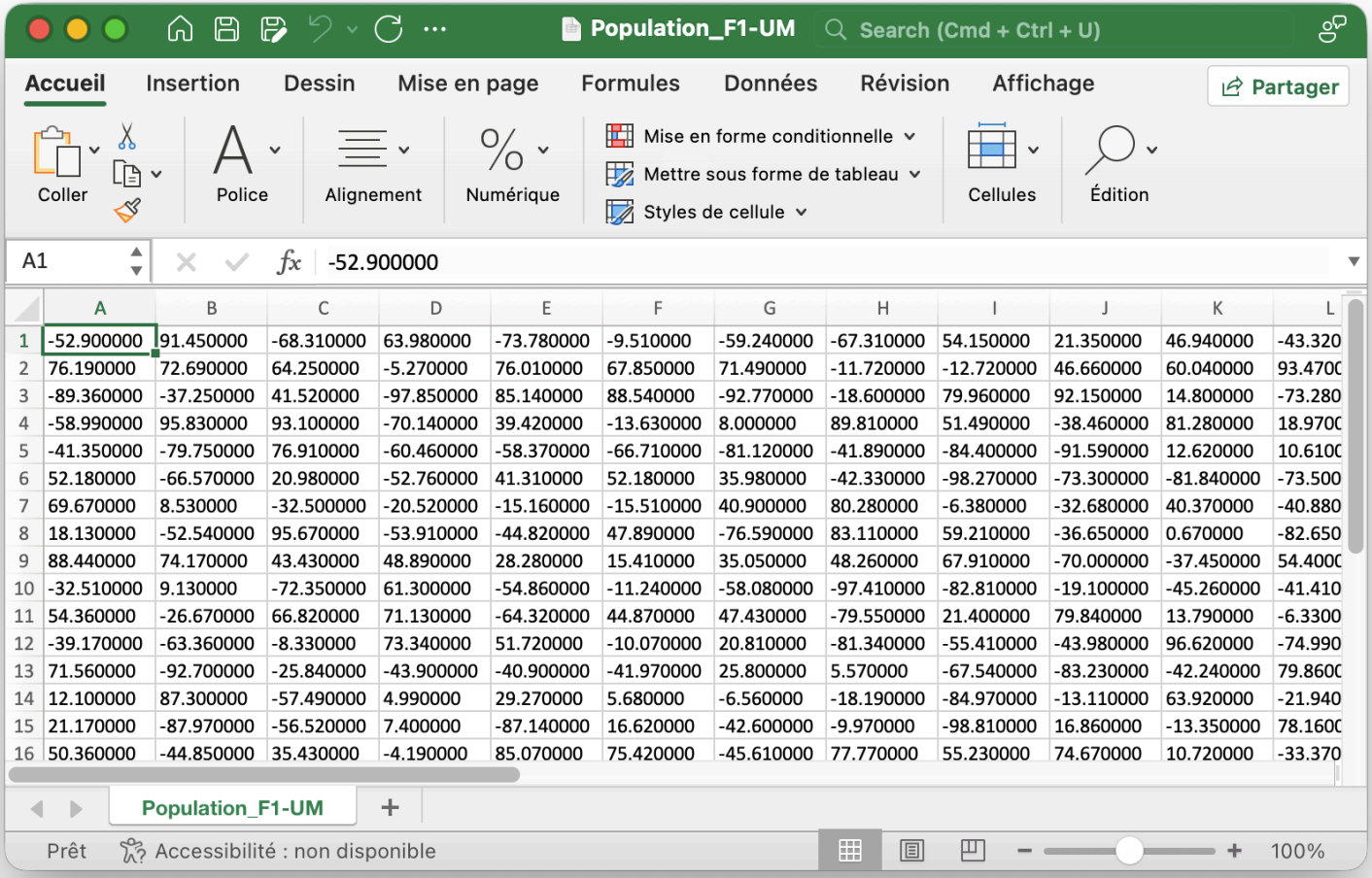
I. Population Initialization

Generation of a population (set) of solutions with two possible options:

- **Random generation**, where **Size** represents the population size (number of solutions to generate).
- **Loading** a **population** of solutions from a **CSV file**.



Example of a CSV file containing a population of solutions generated within the search space of the F1-UM function.

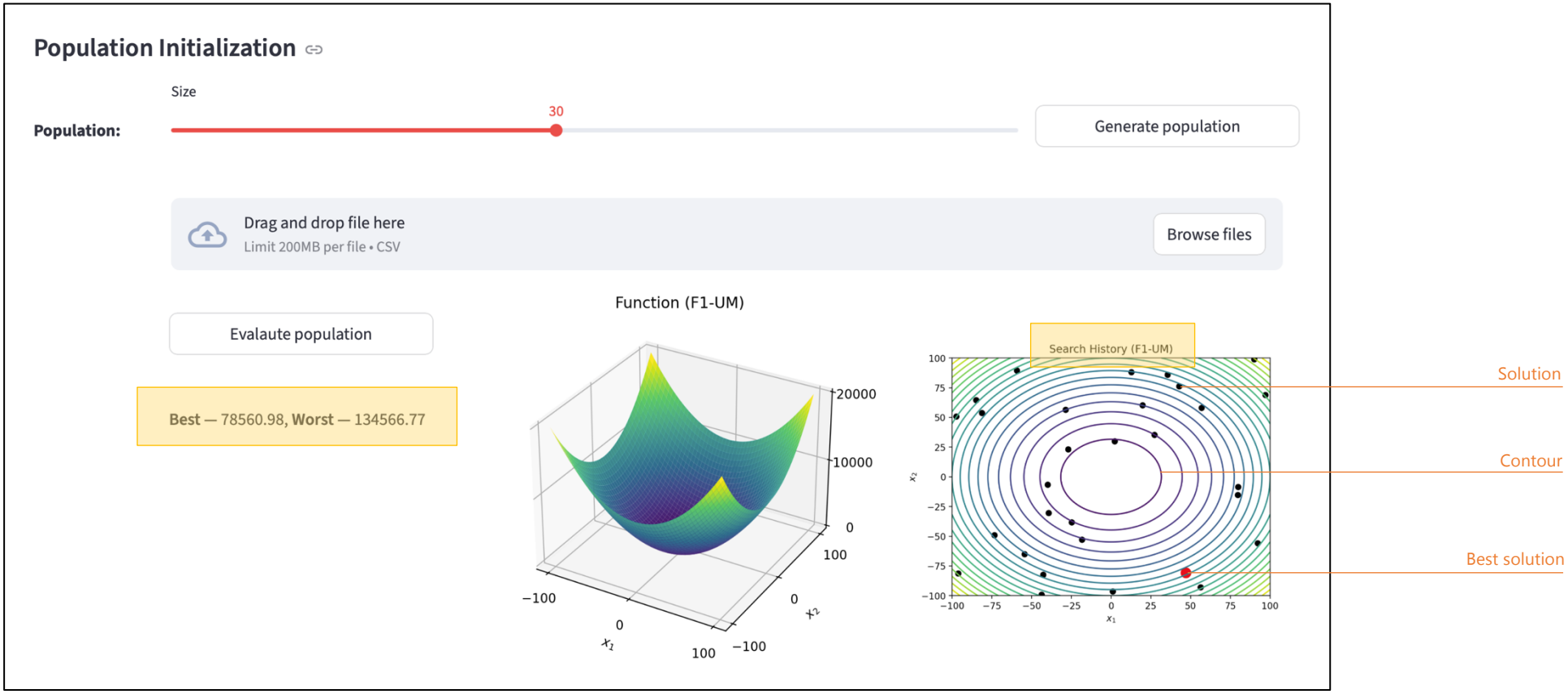


	A	B	C	D	E	F	G	H	I	J	K	L
1	-52.900000	91.450000	-68.310000	63.980000	-73.780000	-9.510000	-59.240000	-67.310000	54.150000	21.350000	46.940000	-43.320
2	76.190000	72.690000	64.250000	-5.270000	76.010000	67.850000	71.490000	-11.720000	-12.720000	46.660000	60.040000	93.4700
3	-89.360000	-37.250000	41.520000	-97.850000	85.140000	88.540000	-92.770000	-18.600000	79.960000	92.150000	14.800000	-73.280
4	-58.990000	95.830000	93.100000	-70.140000	39.420000	-13.630000	8.000000	89.810000	51.490000	-38.460000	81.280000	18.9700
5	-41.350000	-79.750000	76.910000	-60.460000	-58.370000	-66.710000	-81.120000	-41.890000	-84.400000	-91.590000	12.620000	10.6100
6	52.180000	-66.570000	20.980000	-52.760000	41.310000	52.180000	35.980000	-42.330000	-98.270000	-73.300000	-81.840000	-73.500
7	69.670000	8.530000	-32.500000	-20.520000	-15.160000	-15.510000	40.900000	80.280000	-6.380000	-32.680000	40.370000	-40.880
8	18.130000	-52.540000	95.670000	-53.910000	-44.820000	47.890000	-76.590000	83.110000	59.210000	-36.650000	0.670000	-82.650
9	88.440000	74.170000	43.430000	48.890000	28.280000	15.410000	35.050000	48.260000	67.910000	-70.000000	-37.450000	54.4000
10	-32.510000	9.130000	-72.350000	61.300000	-54.860000	-11.240000	-58.080000	-97.410000	-82.810000	-19.100000	-45.260000	-41.410
11	54.360000	-26.670000	66.820000	71.130000	-64.320000	44.870000	47.430000	-79.550000	21.400000	79.840000	13.790000	-6.3300
12	-39.170000	-63.360000	-8.330000	73.340000	51.720000	-10.070000	20.810000	-81.340000	-55.410000	-43.980000	96.620000	-74.990
13	71.560000	-92.700000	-25.840000	-43.900000	-40.900000	-41.970000	25.800000	5.570000	-67.540000	-83.230000	-42.240000	79.8600
14	12.100000	87.300000	-57.490000	4.990000	29.270000	5.680000	-6.560000	-18.190000	-84.970000	-13.110000	63.920000	-21.940
15	21.170000	-87.970000	-56.520000	7.400000	-87.140000	16.620000	-42.600000	-9.970000	-98.810000	16.860000	-13.350000	78.1600
16	50.360000	-44.850000	35.430000	-4.190000	85.070000	75.420000	-45.610000	77.770000	55.230000	74.670000	10.720000	-33.370

II. Population Evaluation

The "Evaluate Population" button displays:

- The best fitness value in the population (the solution that minimizes the objective function).
- The worst fitness value in the population.
- The population solutions and the best solution found so far (**search history**) in two dimensions using a **scatter plot**.



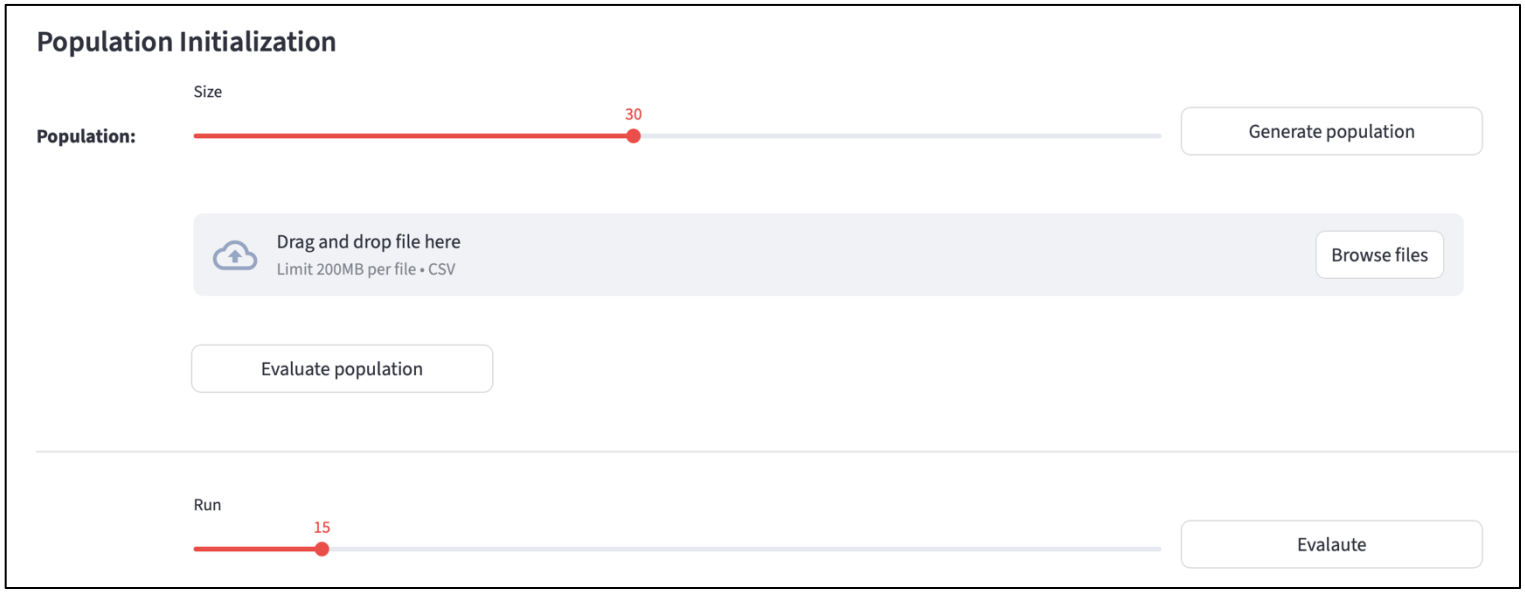
The scatter plot also displays contour lines. The contours are used to:

- **Visualize the structure of the problem:**
  - Where is the minimum located?
  - Is the function convex?
  - Are there multiple local minima?
- **Understand the difficulty of the optimization problem.**
  - For example:
    - Regular circular contours (as in F1) indicate a simple problem.
    - Numerous oscillations (as in F9) indicate a multimodal problem.
    - A narrow valley (as in F5) indicates a problem that is difficult to converge.
- **Observe the behavior of the population:**
  - By adding the population solutions to the scatter plot, we can verify:
    - Whether the solutions are moving toward the global minimum,
    - Whether they are trapped in a local minimum,
    - Their exploration versus exploitation capability.

To display a given solution $x = (x_1,\ x_2, \dots,\ x_D)$ in a scatter plot, we consider only the first two decision variables $x_1$and x_2.

III. Running Multiple Populations

The "Evaluate" button generates randomly multiple populations and returns the results of all the generated populations



The results are calculated as follows :

Metric	Formula	Description	Measure	Goal
Best	$\text{Best} = \min_{1 \le i \le R} B_i$	$\boldsymbol{B_i}$: best fitness value in the population during run i. $\boldsymbol{R}$: number of runs.	Quality solutions	**Smaller Best:** Algorithm can reach high-quality solutions.
Worst	$\text{Worst} = \max_{1 \le i \le R} W_i$	$\boldsymbol{W_i}$: worst fitness value in the population during run i.	Quality solutions	
Mean (average error)	$\text{AVG} = \frac{1}{R}\sum_{i=1}^{R} B_i$		Quality solutions	**Smaller AVG:** Better overall performance.
Standard Deviation	$\text{STD} = \frac{1}{R-1}\sum_{i=1}^{R}(B_i - \text{AVG})^2$		Algorithm stability	**Smaller STD:** Algorithm behaves in a stable manner. **Smaller STD + Larger AVG:** The algorithm is: . Converging too quickly. . Losing diversity (poor exploration). . Getting trapped in a local optimum. **Smaller STD + Smaller AVG:** Ideal balance.

Pseudo-code to Compute Performance Metrics over Multiple Runs

$\boldsymbol{AVG} \leftarrow \mathbf{0}$
For $\boldsymbol{i} \leftarrow \mathbf{1}$ **to** $\boldsymbol{R}$ **do**
  Generate a random population $\boldsymbol{P_i}$ of $\boldsymbol{N}$ solutions $\boldsymbol{x^{(i,1)}}, \boldsymbol{x^{(i,2)}}, \dots, \boldsymbol{x^{(i,1)}}$ where $\boldsymbol{x^{(i,j)}} = \left(\boldsymbol{x_1^{(i,j)}},\ \boldsymbol{x_2^{(i,j)}}, \dots, \boldsymbol{x_D^{(i,j)}}\right)$
  Compute the best fitness $\boldsymbol{B_i}$ and the worst fitness $\boldsymbol{W_i}$ of $\boldsymbol{P_i}$ of run $\boldsymbol{i}$ as follows:
  For $\boldsymbol{j} \leftarrow \mathbf{1}$ **to** $\boldsymbol{N}$ **do**
    Compute $\boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$
    IF $\boldsymbol{j} = \mathbf{1}$ **THEN** $\boldsymbol{B_i} \leftarrow \boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$ **ELSE IF** $\boldsymbol{B_i} > \boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$ **THEN** $\boldsymbol{B_i} \leftarrow \boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$ **END IF; END IF;**
    IF $\boldsymbol{j} = \mathbf{1}$ **THEN** $\boldsymbol{W_i} \leftarrow \boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$ **ELSE IF** $\boldsymbol{W_i} < \boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$ **THEN** $\boldsymbol{W_i} \leftarrow \boldsymbol{f}\left(\boldsymbol{x^{(i,j)}}\right)$ **END IF; END IF;**
  End For;

  Update global best and worst over all runs as follows:
  IF $\boldsymbol{i} = \mathbf{1}$ **THEN** $\mathbf{Best} \leftarrow \boldsymbol{B_i}$ **ELSE IF** $\boldsymbol{B_i} < \mathbf{Best}$ **THEN** $\mathbf{Best} \leftarrow \boldsymbol{B_i}$ **END IF; END IF;**
  IF $\boldsymbol{i} = \mathbf{1}$ **THEN** $\boldsymbol{Worst} \leftarrow \boldsymbol{W_i}$ **ELSE IF** $\boldsymbol{W_i} > \mathbf{Worst}$ **THEN** $\mathbf{Worst} \leftarrow \boldsymbol{W_i}$ **END IF; END IF;**
  $\boldsymbol{AVG} \leftarrow \boldsymbol{AVG} + \boldsymbol{B_i}$
End For;
$\boldsymbol{AVG} \leftarrow \boldsymbol{AVG}/\boldsymbol{R}$

$\boldsymbol{STD}$?